PRISMSPACE – INTELLIGENT MULTI-AGENT PLATFORM

Dept. of Computer Science & Engineering
College of Engineering, Karunagappally

Project Group 01

NOBIN SIJO	KNP23CS082
PRANAV P	KNP23CS085
PRASANTH P	KNP23CS086
SANKAR NARAYAN S	KNP23CS094

Project Coordinator &

Guide

Dr. Leena Silvoster
Associate Professor
College of Engineering, Karunagappally

01. Introduction: PrismSpace Web Platform

- A multi-agent system breaks complex goals into smaller tasks, assigns them to suitable agents, runs independent tasks in parallel, and combines their results.
- **PrismSpace** is a web-based multi-agent intelligence platform designed to coordinate AI agents and manage complex workflows.
- It converts natural-language goals into **Directed Acyclic Graphs (DAGs)** to manage task dependencies and enable parallel execution.
- It provides real-time workflow visualization, local session storage, and human approval controls for sensitive actions.

Problem Statement

Challenges in current AI-agent and workflow systems

02. Problem Statement: Core Challenges in AI Agent Systems

- **Problem:** Coordinating multiple AI agents and complex tasks requires efficient task orchestration, seamless tool integration, clear workflow visibility, and secure execution.
- **Input:** Developer task prompts, workspace and session state, uploaded files, and MCP and human approval configurations.
- **Method:** PrismSpace applies intent classification, supervised model routing, DAG decomposition with topological scheduling, MCP-based tool execution, ORPO preference alignment, and human approval for sensitive actions.
- **Output:** Task results, tool outputs, validation results, and workflow status summaries.

03. Objectives

1. **Intent Classification Module:** Identifies user intent and classifies incoming prompts.
2. **DAG Generation Module:** Decomposes user goals into dependency-aware subtasks and schedules their execution.
3. **Agent Routing Module:** Assigns tasks to suitable agents based on task requirements.
4. **MCP Tool Integration Module:** Connects approved external tools and manages tool execution.
5. **Validation and Human Approval Module:** Validates outputs and requests human approval for sensitive actions.
6. **Workflow Visualization and Session Management Module:** Displays real-time task progress and stores session data locally.

04. Literature Review

Research background, key findings, and identified gap

Multi-Objective Reward Generalization in Deep Reinforcement Learning

Citation	Summary	Advantages & Disadvantages
F. Cornalba, C. Disselkamp, D. Scassola, and C. Helf *Neural Computing and Applications* **Year:** 2024 **Publisher:** Springer Open Access	Investigates **Multi-Objective Deep Reinforcement Learning (MODRL)** for optimizing competing operational objectives (e.g., return vs. risk). Introduces reward generalization techniques that train pareto-optimal policies capable of dynamically adapting to shifting stakeholder preferences without retraining from scratch.	**Advantages:** ▶ Provides structured Pareto-optimal policy exploration across conflicting objectives. ▶ Generalizes across parameterized reward weights without full policy retraining. ▶ Directly informs multi-objective routing (latency vs. cost vs. quality) in PrismSpace. **Disadvantages:** ▶ High initial training sample complexity across multi-dimensional reward spaces. ▶ Evaluated primarily on continuous financial time-series data. ▶ Sensitive to hyperparameter tuning in policy entropy regularization.

Average Reward Adjusted Discounted Reinforcement Learning

Citation	Summary	Advantages & Disadvantages
M. Schneckenreither and G. Moser *Neural Computing and Applications* **Year:** 2025 **Publisher:** Springer Open Access	Proposes **Average-Reward Adjusted Discounted RL (Reinforcement Learning) (ARAD-RL)**, bridging the gap between discounted and average reward criteria in infinite-horizon **MDPs** (Markov Decision Processes). Addresses systematic value estimation bias in standard discounted Q-learning by dynamically subtracting the estimated gain rate during Bellman updates.	**Advantages:** ▶ Eliminates infinite-horizon return explosion while preserving policy convergence. ▶ Improves long-term steady-state throughput in continuous scheduling tasks. ▶ Theoretically sound formulation for continuous agent lifecycle optimization. **Disadvantages:** ▶ Requires accurate empirical estimation of average reward baseline. ▶ Slow convergence in highly non-stationary transition environments. ▶ Adds variance to early-stage **TD** (Temporal Difference) error gradient estimates.

Deep Reinforcement Learning for Multi-Agent Coordination

Citation	Summary	Advantages & Disadvantages
K. Aina and S. Ha *Artificial Life and Robotics* **Year:** 2026 **Publisher:** Springer Open Access	Develops decentralized **MARL** (Multi-Agent Reinforcement Learning) algorithms for multi-agent coordination in constrained environments. Employs **spatial-temporal observation encoding** and localized interaction graphs to coordinate joint movements and resolve deadlocks without requiring centralized global state knowledge.	**Advantages:** ▶ Resolves shared-resource contention and deadlocks via decentralized policies. ▶ Spatial-temporal graph attention scales linearly with agent density. ▶ Applicable to concurrent tool-use contention across worker agents. **Disadvantages:** ▶ Tested in physical robotics grids rather than software **API** (Application Programming Interface) environments. ▶ Vulnerable to communication packet dropouts and sensor latency. ▶ Does not account for semantic language-based coordination.

Open-Ended Coordination Using Modular Open Policies

Citation	Summary	Advantages & Disadvantages
D. Rother, J. Pajarinen, J. Peters, and T. H. Weisswange *Autonomous Agents and Multi-Agent Systems* **Year:** 2025 **Publisher:** Springer Open Access	Presents a framework for **open-ended ad hoc multi-agent coordination**. Utilizes **Modular Open Policies (MOP)** where dynamic agent teams dynamically infer unseen teammate skill representations and synthesize blended coordination strategies without requiring pre-coordinated joint training.	**Advantages:** ▶ Enables zero-shot ad hoc coordination with unfamiliar teammate agent policies. ▶ Modular policy decomposition allows plug-and-play agent role assignment. ▶ Directly adopted in PrismSpace for dynamic worker swarm scaling. **Disadvantages:** ▶ Requires comprehensive pre-trained skill policy libraries for base roles. ▶ Evaluated primarily on benchmark games (Overcooked, **LBF** (Level-Based Foraging)). ▶ Action blending may introduce transient sub-optimal execution steps.

Improving Scalability of **MADRL** (Multi-Agent Deep Reinforcement Learning) with Suboptimal Human Knowledge

Citation	Summary	Advantages & Disadvantages
D. Liu, F. Ren, J. Yan, G. Su, W. Gu, and S. Kato *Autonomous Agents and Multi-Agent Systems* **Year:** 2026 **Publisher:** Springer Open Access	Addresses sample inefficiency in large-scale **Multi-Agent DRL** (Deep Reinforcement Learning) by integrating heuristic human guidance into exploration. Proposes an adaptive filtering mechanism that extracts actionable macro-actions from noisy or sub-optimal human demonstration traces, accelerating multi-agent policy learning.	**Advantages:** ▶ Accelerates policy exploration by up to 4x using noisy demonstration priors. ▶ Robust to suboptimal or conflicting human heuristic rules. ▶ Informs PrismSpace's bootstrap initialization from logged developer sessions. **Disadvantages:** ▶ Quality of learned policies depends on initial heuristic coverage. ▶ Requires complex threshold tuning to transition away from demonstrations. ▶ High memory footprint for maintaining dual replay buffers.

ε-Retraining Reinforcement Learning Algorithms

Citation	Summary	Advantages & Disadvantages
L. Marzari, C. Liu, P. L. Donti, and E. Marchesini *Autonomous Agents and Multi-Agent Systems* **Year:** 2026 **Publisher:** Springer Open Access	Investigates bounded policy fine-tuning through ε-**retraining**, preventing catastrophic forgetting when deep **RL** agents adapt to non-stationary environments. Establishes formal Lipschitz bounds on policy drift, ensuring safe continual learning in deployed agent pipelines.	**Advantages:** ▶ Provides mathematical guarantees against catastrophic policy divergence. ▶ Enables continuous telemetry-driven model fine-tuning in production. ▶ Crucial for stability in PrismSpace's ongoing user interaction learning. **Disadvantages:** ▶ Constrained optimization increases computation per gradient step. ▶ Conservative update bounds may slow adaptation to drastic distribution shifts. ▶ Theoretical guarantees rely on local smoothness assumptions.

Deep Multiagent Reinforcement Learning: Challenges and Directions

Citation	Summary	Advantages & Disadvantages
A. Wong, T. Bäck, A. V. Kononova, and A. Plaat *Artificial Intelligence Review* **Year:** 2023 **Publisher:** Springer Open Access	Surveys **Deep Multiagent Reinforcement Learning (MARL)**, formalizing foundational challenges including non-stationarity, multi-agent credit assignment, and combinatorial action explosion. Evaluates **Centralized Training with Decentralized Execution (CTDE)** and communication architectures for modular orchestration.	**Advantages:** ▶ Comprehensive taxonomy of credit assignment and **CTDE** paradigms. ▶ Establishes formal theoretical criteria for communication protocols. ▶ Provides the theoretical foundation for PrismSpace's modular agent architecture. **Disadvantages:** ▶ Survey paper without empirical experimental evaluation. ▶ Focuses on discrete game simulations rather than **API**-driven software tasks. ▶ Does not cover **LLM** reasoning agents or human-in-the-loop approval gates.

HIAT (Human-in-the-Loop with Auxiliary Tasks): Human-in-the-Loop **RL** with Auxiliary Tasks

Citation	Summary	Advantages & Disadvantages
B. Niu, B. Luo, Y. Cui, X. Xu, Y. Zhao, and Y. Feng *Artificial Intelligence Review* **Year:** 2025 **Publisher:** Springer Open Access	Introduces **HIAT (Human-in-the-Loop with Auxiliary Tasks)**, a framework leveraging human intervention signals as auxiliary self-supervised learning objectives. Models human corrections as feedback gradients that shape internal feature representations, minimizing human intervention frequency over time.	**Advantages:** ▶ Maximizes learning efficiency from scarce human intervention signals. ▶ Significantly reduces human cognitive load through auxiliary reward shaping. ▶ Informs PrismSpace's approval gate telemetry and learning from user edits. **Disadvantages:** ▶ Requires consistent human feedback to avoid biased auxiliary representations. ▶ Intervention timing latency can cause temporal mis-alignment in **RL** updates. ▶ Higher computational overhead during auxiliary task representation training.

ORPO (Odds Ratio Preference Optimization): Monolithic Preference Optimization without Reference Model

Citation	Summary	Advantages & Disadvantages
J. Hong, N. Lee, and J. Thorne *EMNLP* / *arXiv:2403.07691* **Year:** 2024 **Area: LLM** Preference Alignment	**Core Idea (Single-Phase Alignment):** Instead of multi-step **RLHF**, **ORPO** teaches desired answers while penalizing bad answers in one unified equation: $$\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} + \lambda \cdot \mathcal{L}_{\text{OR}}$$ $$\mathcal{L}_{\text{OR}} = -\log \sigma \left(\log \frac{\text{odds}(y_w \mid x)}{\text{odds}(y_l \mid x)} \right)$$ **Key Terms:** ▶ $\mathcal{L}_{\text{SFT}}$: Base supervised fine-tuning loss (correct code/syntax). ▶ $\mathcal{L}_{\text{OR}}, \lambda$: Odds ratio penalty term and penalty weight. ▶ x, y_w, y_l: Prompt, winning action path, and losing action path.	**Advantages:** ▶ Reference-model-free: eliminates maintaining frozen π_{ref} in memory. ▶ Monolithic single-phase training avoids unstable **PPO** policy optimization. ▶ Directly aligns PrismSpace agent trajectories on paired feedback (*chosen* vs. *rejected*). **Disadvantages:** ▶ Requires high-quality pairwise preference datasets (y_w, y_l). ▶ Sensitive to λ hyperparameter balance.

Reinforcement Online Learning to Rank with Unbiased Reward Shaping

Citation	Summary	Advantages & Disadvantages
S. Zhuang, Z. Qiao, and G. Zuccon *Information Retrieval Journal* **Year:** 2023 **Publisher:** Springer Open Access	Develops an **Online Learning to Rank (OLTR)** framework using counterfactual reinforcement learning. Employs **inverse propensity weighting (IPW)** and unbiased reward shaping to correct for position, presentation, and click bias in user interaction logs during continuous ranker optimization.	**Advantages:** ▶ Guarantees unbiased gradient estimation from observational implicit feedback. ▶ Operates entirely online without requiring manual document relevance labels. ▶ Directly informs PrismSpace's context retrieval ranking feedback loop. **Disadvantages:** ▶ High propensity score variance can destabilize gradient updates. ▶ Requires large volumes of query-click logs to converge. ▶ Assumes position-based user click models that may not hold in code search.

Knowledge Learning for Securing Workflow Scheduling in **MEC** (Mobile Edge Computing)

Citation	Summary	Advantages & Disadvantages
T. M. Ghazal, A. E. Awouda, M. K. Hasan, A. H. A. Rahman, S. Islam, R. A. Saeed, H. Elshafie, A. Iqbal, and A. H. Hussein *Mobile Networks and Applications* **Year:** 2025 **Publisher:** Springer Open Access	Proposes a **Knowledge-Guided Reinforcement Learning (KG-RL)** scheduler for complex workflow **DAGs** in distributed edge-cloud environments. Integrates security constraint validation directly into the state transition model, preventing insecure task placement while optimizing makespan.	**Advantages:** ▶ Enforces strict security boundary constraints during automated task scheduling. ▶ Effectively handles multi-dependency Directed Acyclic Graph (**DAG**) workflows. ▶ Informs PrismSpace's sandboxed tool execution and dependency resolution. **Disadvantages:** ▶ Constraint checking introduces scheduling latency for massive **DAGs**. ▶ Assumes static security classification matrices across network nodes. ▶ Limited evaluation on non-deterministic tool failure scenarios.

Energy-Aware Cooperative Multi-Fitness Evolutionary Workflow Scheduling

Citation	Summary	Advantages & Disadvantages
P. Barredo and J. Puente *Natural Computing* **Year:** 2025 **Publisher:** Springer Open Access	Presents a **Cooperative Multi-Fitness Evolutionary Algorithm (CMFEA)** for multi-objective workflow scheduling. Optimizes trade-offs between execution time, monetary compute cost, and energy consumption using island-model cooperative coevolution across heterogeneous worker pools.	**Advantages:** ▶ Pareto-optimal trade-offs across makespan, financial cost, and energy. ▶ Cooperative sub-population decomposition prevents premature local convergence. ▶ Provides heuristics for PrismSpace's multi-agent model provider selection. **Disadvantages:** ▶ Evolutionary search has higher runtime latency than single-pass ML heuristics. ▶ Cannot adapt instantaneously to real-time **API** rate limit drops. ▶ Requires parameter tuning across mutation, crossover, and migration rates.

Optimized Task Scheduling Using Hybrid **DL** (Deep Learning) and Metaheuristics

Citation	Summary	Advantages & Disadvantages
A. Heirati, M. Fartash, and M. Khalily-Dermany *Neural Processing Letters* **Year:** 2026 **Publisher:** Springer Open Access	Combines **Deep Learning (LSTM (Long Short-Term Memory))** workload prediction with metaheuristic optimization (Harris Hawks Optimization) for proactive workflow dispatching. Predicts task execution durations and memory bottlenecks in advance to prevent node overloading and minimize queue waiting times.	**Advantages:** ▶ Proactive workload forecasting prevents sudden compute node thrashing. ▶ Hybrid pipeline balances search exploration with predictive dispatching. ▶ Relevant for predicting token latency and cost before agent invocation. **Disadvantages:** ▶ Two-stage pipeline introduces inference overhead before scheduling decisions. ▶ **LSTM** prediction accuracy degrades under sudden irregular workload spikes. ▶ Complex offline model retraining requirements.

Cost-Aware Workflow Offloading in Edge-Cloud Computing via **GA** (Genetic Algorithm)

Citation	Summary	Advantages & Disadvantages
S. Abdi, M. Ashjaei, and S. Mubeen *The Journal of Supercomputing* **Year:** 2024 **Publisher:** Springer Open Access	Proposes a **Genetic Algorithm-based workflow offloading** framework considering heterogeneous network bandwidth, deadline constraints, and monetary costs. Implements chromosome repair operators to preserve **DAG** precedence validity throughout mutation and crossover.	**Advantages:** ▶ Guarantees valid **DAG** dependency structures via specialized repair operators. ▶ Explicitly accounts for network transfer latency between pipeline stages. ▶ Informs subtask payload transfer between worker agents in PrismSpace. **Disadvantages:** ▶ Computationally heavy for interactive real-time user query response. ▶ Assumes deterministic network transfer bandwidth in cloud environments. ▶ Does not model dynamic agent failures or retries.

Dynamic Role Discovery and Assignment in Multi-Agent Task Decomposition

Citation	Summary	Advantages & Disadvantages
Y. Xia, J. Zhu, and L. Zhu *Complex & Intelligent Systems* **Year:** 2023 **Publisher:** Springer Open Access	Proposes **dynamic role discovery and assignment** that decomposes complex high-level objectives into lower-level subtasks. Uses learned persona embeddings and mutual information to adaptively allocate subtasks according to situational agent capabilities, eliminating rigid static assignments.	**Advantages:** ▶ Enables flexible subtask allocation without hardcoded role hierarchies. ▶ Hierarchical decomposition prevents single-agent cognitive context overload. ▶ Serves as core theoretical basis for PrismSpace's dynamic agent routing. **Disadvantages:** ▶ Role discovery clustering introduces upfront computational overhead. ▶ Performance depends on representation vector accuracy for subtasks. ▶ Evaluated primarily on synthetic collaborative coordination benchmarks.

Decision Variable Contribution Based Adaptive Multi-Objective Scheduling

Citation	Summary	Advantages & Disadvantages
J. Li, L. Xing, W. Zhong, Z. Cai, and F. Hou *Complex & Intelligent Systems* **Year:** 2023 **Publisher:** Springer Open Access	Introduces a **Decision Variable Contribution (DVC)** adaptive mechanism for evolutionary multi-objective workflow scheduling. Evaluates the differential sensitivity of individual decision variables to focus evolutionary search operators on high-impact scheduling bottlenecks.	**Advantages:** ▶ Accelerates Pareto-front convergence by prioritizing high-variance variables. ▶ Dynamically adapts search intensity across workflow bottleneck stages. ▶ Useful for pinpointing high-latency critical path tools in agent workflows. **Disadvantages:** ▶ Sensitivity calculation adds per-generation matrix computation cost. ▶ Can over-concentrate on dominant objectives at the expense of diversity. ▶ Requires calibrated normalization across widely different metric units.

GHQ: Grouped Hybrid Q-Learning for Heterogeneous Multi-Agent RL

Citation	Summary	Advantages & Disadvantages
X. Yu, Y. Lin, X. Wang, S. Han, and K. Lv *Complex & Intelligent Systems* **Year:** 2024 **Publisher:** Springer Open Access	Develops **Grouped Hybrid Q-learning (GHQ)** for cooperative heterogeneous multi-agent systems. Clusters agents with similar capabilities into functional cohorts that share value approximations while maintaining decoupled action execution heads for role specialization.	**Advantages:** ▶ Scales value estimation across large heterogeneous swarms via cohort grouping. ▶ Reduces joint action space dimensionality from exponential to block-linear. ▶ Directly applicable to categorizing **LLM** worker agent skill specializations. **Disadvantages:** ▶ Cohort clustering boundaries can be rigid if agent capabilities evolve. ▶ Intra-group credit assignment remains non-trivial in dense interactions. ▶ Requires domain knowledge to establish initial grouping heuristics.

Decentralized Multi-Agent Cooperation via Adaptive Partner Modeling

Citation	Summary	Advantages & Disadvantages
C. Xu, J. Wang, X. Zhu, Y. Yue, W. Zhou, Z. Liang, and D. Wojtczak *Complex & Intelligent Systems* **Year:** 2024 **Publisher:** Springer Open Access	Tackles environmental non-stationarity in decentralized multi-agent cooperation using **Adaptive Partner Modeling (APM)**. Instead of relying on expensive continuous inter-agent broadcast communication, agents infer partner policies and intended actions from local observation histories.	**Advantages:** ▶ Minimizes inter-agent communication bandwidth to near zero during execution. ▶ Stabilizes decentralized joint execution despite teammate policy shifts. ▶ Directly adopted in PrismSpace for zero-communication agent coordination. **Disadvantages:** ▶ Partner inference error propagates if historical observations are noisy. ▶ Inference networks introduce local computational and memory overhead. ▶ Assumes partners follow relatively stationary parameterized policy spaces.

A Responsible AI Framework: Pipeline Contextualisation

Citation	Summary	Advantages & Disadvantages
E. Vyhmeister, G. G. Castañé, P.-O. Östberg, and S. Thevenin *AI and Ethics* **Year:** 2023 **Publisher:** Springer Open Access	Formulates an operational **Responsible AI Framework** providing pipeline contextualisation across the full model lifecycle. Specifies auditable checkpoints, data lineage tracking, and bias mitigation protocols for autonomous AI decision-making systems.	**Advantages:** ▶ Holistic governance framework linking technical architecture to ethical safety. ▶ Defines concrete telemetry points for data provenance and action auditing. ▶ Guides PrismSpace's execution audit logging and human inspection UI. **Disadvantages:** ▶ Qualitative framework that requires custom translation into concrete code gates. ▶ Audit logging introduces storage overhead and slight execution latency. ▶ Does not provide automated formal verification of **LLM** reasoning traces.

Meaningful Human Control: Actionable Properties for AI Development

Citation	Summary	Advantages & Disadvantages
L. Cavalcante Siebert, M. L. Lupetti, E. Aizenberg, N. Beckers, A. Zgonnikov, H. Veluwenkamp, D. Abbink, E. Giaccardi, G.-J. Houben, C. M. Jonker, J. van den Hoven, D. Forster, and R. L. Lagendijk *AI and Ethics* **Year:** 2023 **Publisher:** Springer Open Access	Formalizes the philosophical concept of **Meaningful Human Control (MHC)** into actionable software design properties: **Tracking** (system responsive to human moral reasons) and **Tracing** (system actions causally attributable to accountable human supervisors).	**Advantages:** ▶ Translates abstract governance principles into rigorous software specifications. ▶ Establishes tracking and tracing requirements for autonomous tool execution. ▶ Directly guides the design of PrismSpace's Human-in-the-Loop (**HITL**) approval gates. **Disadvantages:** ▶ High human intervention frequency can degrade automated agent throughput. ▶ Defining actionable reason boundaries requires careful domain calibration. ▶ Assumes human supervisor maintains full comprehension of complex agent plans.

CPN-HRL (Cloud-Private-Network Hierarchical Reinforcement Learning): Hierarchical **DRL** for Priority-Aware Task Scheduling

Citation	Summary	Advantages & Disadvantages
Q. Yue, L. Tian, X. Feng, Z. Yuan, S. Wei, J. Xia, B. Chen, X. Song, and Y. Yao *Journal of King Saud University CIS* **Year:** 2026 **Publisher:** Springer Open Access	Presents **CPN-HRL**, a Hierarchical Reinforcement Learning framework for priority-aware task scheduling in Cloud-Edge-End networks. Uses a macro-policy to partition task **DAGs** by priority and latency budgets, while micro-policies schedule subtasks onto optimal compute nodes.	**Advantages:** ▶ Hierarchical policy decomposition effectively handles complex task **DAGs**. ▶ Guarantees priority-aware quality of service under tight latency deadlines. ▶ Serves as primary reference for PrismSpace's **DAG** workflow scheduling engine. **Disadvantages:** ▶ Multi-tier policy architecture requires synchronous training coordination. ▶ Edge node failure requires immediate micro-policy replanning. ▶ Evaluated in simulated telecom cloud-edge topologies.

Automatic Discovery and Description of Human Planning Strategies

Citation	Summary	Advantages & Disadvantages
J. Skirzyński, Y. Jain, and F. Lieder *Behavior Research Methods* **Year:** 2023 **Publisher:** Springer Open Access	Develops an algorithmic framework for **automatically discovering and formalizing planning strategies** from human behavior traces. Models task planning as resource-rational search over Directed Graphical Models, producing interpretable decision trees that characterize human heuristic planning.	**Advantages:** ▶ Extracts interpretable, human-like heuristic planning strategies automatically. ▶ Quantifies trade-offs between cognitive search effort and decision quality. ▶ Informs PrismSpace's hierarchical prompt decomposition and agent planning. **Disadvantages:** ▶ Search space for strategy discovery grows exponentially with problem depth. ▶ Human behavioral experiments conducted in stylized cognitive testbeds. ▶ Static decision trees struggle to capture fluid **LLM** reasoning trajectories.

Data-Intensive Workflow Scheduling Based on **DRL** in Multi-Clouds

Citation	Summary	Advantages & Disadvantages
S. Zhang, Z. Zhao, C. Liu, and S. Qin *Journal of Cloud Computing* **Year:** 2023 **Publisher:** Springer Open Access	Proposes a **Deep Q-Network (DQN)** scheduler optimized for data-intensive workflows across multi-cloud environments. Explicitly models dataset placement and cross-datacenter transfer latency to minimize both workflow completion time and data egress billing.	**Advantages:** ▶ Optimizes data locality to prevent massive cross-cloud egress overhead. ▶ Adaptive reward function balances monetary cost and makespan constraints. ▶ Directly relevant to caching and embedding retrieval across distributed agents. **Disadvantages:** ▶ Assumes deterministic file sizes and network bandwidth availability. ▶ Q-network value overestimation requires double-**DQN** correction. ▶ State representation size scales with the total number of cloud datacenters.

RL-Based Task Scheduling for Sustainable Federated Cloud Computing

Citation	Summary	Advantages & Disadvantages
Z. Wang, S. Chen, L. Bai, J. Gao, J. Tao, R. R. Bond, and M. D. Mulvenna *Journal of Cloud Computing* **Year:** 2023 **Publisher:** Springer Open Access	Develops an **Actor-Critic RL scheduling framework** for federated cloud datacenters that incorporates renewable energy availability into scheduling policies. Dynamically routes compute tasks to datacenters with green energy surpluses without violating user SLA deadlines.	**Advantages:** ▶ Incorporates environmental sustainability and carbon metrics into scheduling. ▶ Actor-Critic policy gradient handles continuous action spaces effectively. ▶ Provides conceptual foundation for multi-attribute provider routing. **Disadvantages:** ▶ Requires real-time telemetry from green energy generation sources. ▶ Trade-off between carbon footprint and network transmission latency. ▶ Sensitive to reward scaling factors between SLA penalties and green rewards.

Hybrid Dolphin Swarm Sparrow Search Multi-Objective Workflow Scheduling

Citation	Summary	Advantages & Disadvantages
S. Subashree, M. Rajakumaran, G. Pushpa, and S. Senthilkumar *Journal of Cloud Computing* **Year:** 2025 **Publisher:** Springer Open Access	Proposes a hybrid bio-inspired metaheuristic combining **Dolphin Swarm Optimization (DSA)** and **Sparrow Search Algorithm (SSA)** for multi-objective cloud workflow scheduling. Optimizes execution makespan, virtual machine utilization, and monetary execution expenditure.	**Advantages:** ▶ Combines global exploration of DSA with local exploitation of SSA. ▶ Demonstrates superior Pareto hypervolume compared to standard **GA** and **PSO** (Particle Swarm Optimization). ▶ Provides strong comparative baselines for workflow optimization algorithms. **Disadvantages:** ▶ Metaheuristic iteration runtime is too slow for sub-second reactive routing. ▶ Multiple tuning parameters (predation rate, vocalization index) required. ▶ Lacks theoretical convergence rate bounds for stochastic optimization.

Research Gap

What the surveyed methods already do

MARL works address **CTDE**, role grouping, and ad-hoc coordination. **ORPO** aligns **LLM** preferences without reference models, while **HIAT** leverages human interventions as auxiliary **RL** signals. **CPN-HRL** and **DAG** schedulers optimize cloud task allocation, and **MHC** formalizes tracking and tracing properties.

Limitations observed in those papers

Coordination benchmarks focus on gaming (**SMAC**, Overcooked) rather than filesystem/**API** tooling. **ORPO** lacks grounding in **MCP ReAct** tool traces, while **HIAT** targets continuous robotic control instead of code execution. Cloud schedulers assume static **DAGs** or slow metaheuristics ill-suited for real-time **LLM** routing.

Why PrismSpace is required

No reviewed system jointly grounds tool observations, aligns policies on agent trajectories, dynamically routes heterogeneous **LLMs**, and enforces human-in-the-loop safety gates within a unified developer workspace.

03. Comparative Analysis: Existing Platforms vs. PrismSpace

Simple comparison: Comparing PrismSpace with conversational AI, autonomous-agent, and multi-agent platforms.

Capability Dimension	ChatGPT Web (OpenAI)	AutoGPT (Autonomous Loop)	CrewAI (Multi-Agent Swarm)	PrismSpace (Ours) Local Multi-Agent Platform
Execution Topology	One conversation at a time	Repeats steps automatically; may not stop	Agents follow a fixed sequence	**DAG (Directed Acyclic Graph) Scheduling:** Runs dependent tasks in parallel, with up to three workers
Model Strategy & Routing	Mostly one provider and model	Usually one model with custom scripts	Model selected for each agent role	**Multi-Model Router:** Selects a suitable provider using task needs, speed, and cost
Tool Connectivity Protocol	Built-in plugins and function calls	Custom scripts and shell commands	Tool wrappers with fixed formats	**MCP (Model Context Protocol):** A common interface for tools such as Gmail, Drive, files, and web services
Observability & Streaming	Shows answer text; internal steps are hidden	Terminal logs only	Text traces from agent execution	**Live Run Status + Workflow Graph:** Shows task progress and each node's state
Safety & Quality Control	Provider filters; no user approval step	Fixed limits; limited output checking	Agent reviews its own output	**HITL (Human-in-the-Loop) + ORPO (Odds Ratio Preference Optimization):** Pauses risky tasks and ranks preferred outputs
Persistence & Privacy	Data stored in the provider's cloud	Local files without a structured database	Cloud or temporary in-memory state	**Local-First PrismDB:** Stores sessions and files in the browser's IndexedDB database

Algorithm

Step-by-step execution of the PrismSpace multi-agent workflow

Algorithm: Step 1 – Understand the User Prompt

Step 1: Prompt Parsing and Structured Representation

1.1 Read the user's natural-language prompt and identify the main goal.

1.2 Retrieve persistent session context and workspace history from browser IndexedDB (`PrismDB`):

- **1.2.1** Load relevant conversation history, saved preferences, and agent state.
- **1.2.2** Resolve referenced files, repositories, and available permissions.

1.3 Build a structured task record containing the goal, constraints, context, allowed tools, and expected output.

- **1.3.1** Record deadlines, output format, and other task constraints.
- **1.3.2** Attach the MCP tools and permissions required for execution.

1.4 Perform input boundary and security screening:

- **1.4.1** Check input size and screen for prompt-injection patterns.
- **1.4.2** Mark file, email, or other state-changing actions for user approval.

Algorithm: Step 2 – Classify the Request

Step 2: Intent Classification and Capability Check

2.1 Classify the prompt into a functional domain such as automation, document work, research, or coding:

- **2.1.1** Extract semantic and keyword features from the prompt.
- **2.1.2** Use the intent classifier to select the most likely domain.

2.2 Profile required system capabilities and operational risk levels:

- **2.2.1** Identify required resources, such as web search, file access, or terminal execution.
- **2.2.2** Separate read-only work from actions that need approval.

2.3 Select optimal execution pathway:

- **2.3.1** Send simple requests directly to one model.
- **2.3.2** Send complex requests to the DAG planner for decomposition.

Step 3: DAG Decomposition and Model Routing

3.1 Deconstruct objective into Directed Acyclic Graph $\mathcal{G} = (V, E)$:

- **3.1.1** Create one vertex for each atomic task and define its expected output.
- **3.1.2** Connect tasks with directed edges when one task depends on another's output.
- **3.1.3** Hold a dependent task until all required inputs are available.

3.2 Compute topological task execution schedule $\pi = \mathrm{TopologicalSort}(\mathcal{G})$:

- **3.2.1** Find independent tasks that can run in parallel ($K \leq 3$).
- **3.2.2** Release dependent tasks when their parent tasks finish.

3.3 Select a model for each sub-task by balancing cost, latency, and quality:

$$\text{Provider}^* = \arg\min_m \left(\lambda_{\text{cost}} \, \text{Cost}_m + \lambda_{\text{latency}} \, \text{Latency}_m \right) \quad \text{s.t. } \text{Quality}_m \geq \tau$$

Meaning: Choose the lowest-cost and lowest-latency model that still meets the required quality threshold.

- **3.3.1** Use fast backends for simple or latency-sensitive tasks.
- **3.3.2** Use stronger models for reasoning, research, and code generation.

Step 4: Role-Based Task Assignment and Dispatching

4.1 Check the specialist-agent registry and their capability profiles:

- **4.1.1** Review each agent's role, instructions, and assigned tools.
- **4.1.2** Check worker availability before dispatch.

4.2 Match each sub-task to the agent with the closest capability profile:

$$\text{Agent}^* = \arg\max_{a \in \mathcal{A}} \ \text{CosineSimilarity}\big(\mathbf{Req}(v_i),\, \mathbf{Cap}(a)\big)$$

Meaning: Compare the task requirements with each agent's capabilities and choose the best match.

- **4.2.1** Send information-gathering tasks to the Researcher.
- **4.2.2** Send software tasks to the Coder.
- **4.2.3** Send checking and policy tasks to the Validator.

4.3 Manage task lifecycle states in dispatcher queue:

- **4.3.1** Keep tasks in `PENDING` until their dependencies finish.
- **4.3.2** Move ready tasks to `READY` and dispatch them.

Step 5: Agent Decision Cycle and Inter-Agent Communication

5.1 Run a ReAct cycle for each dispatched agent:

$$\text{State } s_t \longrightarrow \textbf{Observe} \longrightarrow \textbf{Reason} \longrightarrow \textbf{Act} \longrightarrow \textbf{Reflect}$$

Meaning: Observe the task, reason about the next step, act with a tool or response, and reflect on the result.

- **5.1.1** **Observe:** Read the task, constraints, and parent outputs.
- **5.1.2** **Reason:** Choose the next operation.
- **5.1.3** **Act:** Call an MCP tool or generate an answer.
- **5.1.4** **Reflect:** Check the result and correct local errors.

5.2 Transmit intermediate artifacts via DAG edge message passing:

- **5.2.1** Pass verified outputs from completed tasks to their dependent tasks.
- **5.2.2** Pass only relevant artifacts to limit context growth.

5.3 Synchronize shared state via local blackboard memory:

- **5.3.1** Save task state, conversation turns, and file references in `PrismDB`.
- **5.3.2** Notify dependent workers when a task completes.

Step 6: Specialist Execution Workflows – Planner & Assistant

6.1 Execute Planner Agent workflow (System Architect & Coordinator):

- **6.1.1** Read the objective and relevant context from `PrismDB`.
- **6.1.2** Split the objective into independent and dependent sub-goals.
- **6.1.3** Build the DAG and add capability tags.
- **6.1.4** Select models and enqueue the topological schedule.

6.2 Execute Assistant Agent workflow (Productivity & Communication):

- **6.2.1** Retrieve preferences and reference documents.
- **6.2.2** Create a structured draft or summary.
- **6.2.3** Use Google Workspace tools when required.
- **6.2.4** Verify the draft and request approval before sending it.

Step 7: Specialist Execution Workflows – Researcher, Coder & Validator

7.1 Execute Researcher Agent workflow (Information Retrieval & Synthesis):
- **7.1.1** Create search queries and use the Browser MCP server.
- **7.1.2** Extract relevant information from retrieved documents.
- **7.1.3** Remove duplicates and prepare a cited report.

7.2 Execute Coder Agent workflow (Software Synthesis & Execution):
- **7.2.1** Inspect the repository and edit the required source files.
- **7.2.2** Run tests or scripts in the terminal and inspect errors.
- **7.2.3** Fix syntax and runtime problems using test feedback.

7.3 Execute Validator Agent workflow (Quality Auditing & Safety Gate):
- **7.3.1** Check outputs against safety, format, and functional requirements.
- **7.3.2** Return **PASS** for valid output or **FAIL** with diagnostics.

Step 8: Standardized Tool Execution via Model Context Protocol

8.1 Interface agents to external tools via Model Context Protocol (MCP):

- **8.1.1** Use JSON-RPC for consistent client-server communication.
- **8.1.2** Discover tool schemas, inputs, and return types at runtime.
- **8.1.3** Keep agent logic independent of tool implementation details.

8.2 Orchestrate multi-domain tool execution ecosystem:

- **8.2.1** **System & Dev Tools:** Filesystem, terminal, and GitHub.
- **8.2.2** **Productivity Tools:** Gmail and Google Drive.
- **8.2.3** **Web Intelligence:** Browser search and document retrieval.

8.3 Enforce Human-in-the-Loop (HITL) authorization policy:

- **8.3.1** Allow read-only queries to run automatically.
- **8.3.2** Request approval for file changes, email sending, or other state changes.

Step 9: Context Compaction and Shared Memory Management

9.1 Address context window constraints and token consumption limits:

- **9.1.1** Keep the active context within the model's token limit.
- **9.1.2** Reduce unnecessary history to control latency and attention cost.

9.2 Compact the context by keeping verified outputs and removing stale or repeated content.

- **9.2.1** Forward only verified parent outputs to dependent tasks.
- **9.2.2** Remove redundant logs, drafts, and intermediate reasoning.

9.3 Persist global execution state using client-side database (`PrismDB`):

- **9.3.1** Save session turns, task graphs, and tool logs in IndexedDB.
- **9.3.2** Preserve local state for privacy and recovery after failure.

Step 10: Single-Stage Preference Alignment (ORPO) and Verification

10.1 Train the response-quality model with chosen and rejected examples using ORPO:

$$\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} + \lambda_{\text{OR}} \cdot \mathcal{L}_{\text{OR}} \quad \text{where} \quad \mathcal{L}_{\text{OR}} = -\log \sigma \left(\log \left[\frac{\text{odds}_\theta(y_w \mid x)}{\text{odds}_\theta(y_l \mid x)} \right] \right)$$

Meaning: Combine normal supervised loss with a preference loss that favors the chosen response y_w over the rejected response y_l.

10.1.1 Increase the likelihood of chosen responses and reduce the likelihood of rejected responses.

10.1.2 Train on preference pairs (x, y_w, y_l) without a separate reward model.

10.2 Apply a quality gate before accepting an output:

10.2.1 Score each deliverable with the fine-tuned preference model.

10.2.2 Check instruction following, format, and safety requirements.

10.3 Handle quality failures via targeted DAG re-planning:

10.3.1 Record errors when an output fails the quality threshold.

10.3.2 Send the failed branch back to the planner for targeted revision.

Step 11: Artifact Assembly and Global State Commit

11.1 Aggregate and assemble multi-agent deliverables:

- **11.1.1** Collect verified outputs from completed DAG tasks.
- **11.1.2** Combine reports, code changes, and formatted content into one deliverable.

11.2 Evaluate the final deliverable for correctness, completeness, and timeliness:

- **11.2.1** Reject or revise the deliverable if a required dimension is weak.
- **11.2.2** Log quality results to improve future routing decisions.

11.3 Save the completed session state:

- **11.3.1** Write conversation records, DAG state, and tool receipts to `PrismDB`.
- **11.3.2** Record latency, token use, and audit information.

Step 12: Output Delivery and Reactive Workspace Presentation

12.1 Synthesize and package final deliverable artifacts:

- **12.1.1** Package reports, verified code patches, and document drafts.
- **12.1.2** Create an audit receipt with agent, timing, and tool information.

12.2 Render presentation layer via multi-pane workspace interface:

- **12.2.1** Show task states such as `Queued`, `Running`, `Success`, and `Failed`.
- **12.2.2** Render the result with formatting, syntax highlighting, and export controls.
- **12.2.3** Show approval controls for pending external actions.

12.3 Synchronize client state reactively via local database transactions:

- **12.3.1** Save verified state changes and output artifacts in `PrismDB`.
- **12.3.2** Update the interface when local database state changes.

Project Architecture, Working & System Design

Project workflow, architecture, working process, agent lifecycle, dataset pipeline, DAG scheduling, and flow diagrams

06. PrismSpace Workflow (Figure 1.0)

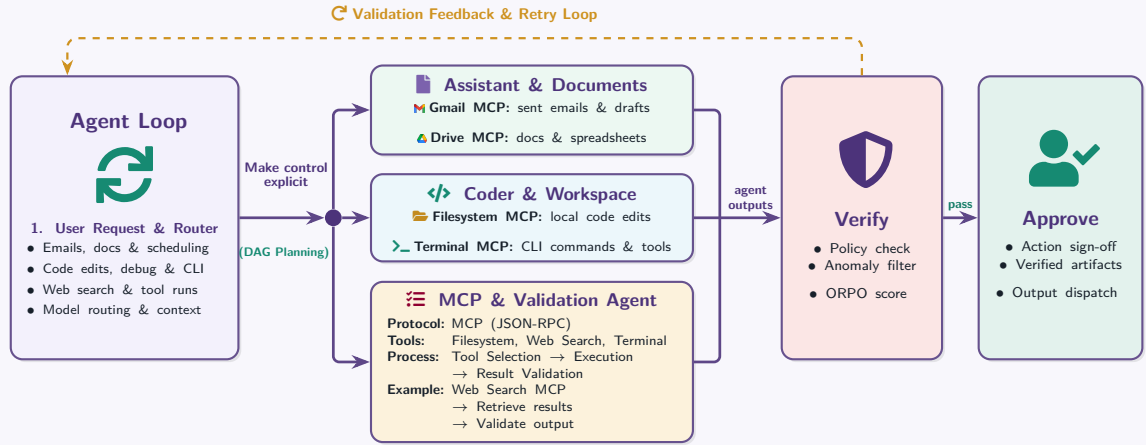


Figure 1.0: PrismSpace Workflow | **DAG** = Directed Acyclic Graph | **MCP** = Model Context Protocol | **ReAct** = Reason + Act | **ORPO** = Odds Ratio Preference Optimization | **HITL** = Human-in-the-Loop

10. Unified Dataset Preparation & Multi-Model Training Pipeline

Raw multi-source datasets flow through normalization and curation into four supervised tables, then train eleven production models.

1. Raw Datasets

- **Intent corpora** – task / tool labels
  agent traces, `AgentInstruct`, `APIBench`, `BEIR`, `BFCL`
- **Agent trajectories** – task success
- **RouterBench** – provider choice
- **UltraFeedback** – preference pairs

`.parquet, .json, .jsonl`

2. Supervised Tables: Fields & Training Roles

Dataset → Table	Training fields	Algorithm	Resultant model
Intent corpora `intent.jsonl`	task prompt; intent label; tool / API cues; agent type; BEIR relevance / research intent	TF–IDF / embeddings + XGBoost classifier.	IntentClassifier
Trajectories `success.jsonl`	prompt / task trace; step timings; tool calls; reward (0/1)	XGBoost binary classifier.	SuccessPredictor
RouterBench `provider.jsonl`	prompt; winner / provider target; vote_score	XGBoost ranking/classifier.	ModelRouter
UltraFeedback `reward.jsonl`	prompt; chosen response; rejected response; preference pair	ORPO (Odds Ratio Preference Optimization).	RewardModel

IntentClassifier
SuccessPredictor
ModelRouter
RewardModel

3. Resultant Production Models

- IntentClassifier
- SuccessPredictor
- ModelRouter
- RewardModel

Dataset split: 80% training | 10% validation | 10% testing.
The held-out 20% is divided equally for validation and final testing.

10. Model Training Map: Algorithm Choice by Responsibility

Model-to-Algorithm Training Matrix

Model	Type	ML algorithm for training	Why choose this algorithm?
Intent Classifier	Classification	XGBoost classifier	Helps understand what the user is asking for and places each request into the appropriate task category.
Agent Router	Multilabel	Binary-relevance XGBoost	Predicts several specialist agents independently while preserving strong tabular/text-feature performance.
Provider Router	Classification	XGBoost ranking/classifier	**XGBoost classifier:** filters suitable providers; **XGBoost ranking:** orders them using capability, quality, and availability.
Reward ORPO Adapter	ORPO	ORPO + Reinforcement Learning	**ORPO (Odds Ratio Preference Optimization):** aligns language models using preference data. It is a reward-model-free alternative that eliminates the need for a separate reward model or reference model.
FAISS Retrieval Index	Unsupervised	Sentence embeddings + FAISS	**Sentence embeddings:** convert text into meaning-based vectors; **FAISS:** performs fast nearest-neighbor search without class labels.

09. Model Responsibilities & Inference Flow



Intent Classifier: Prompt Decomposition, NLP Algorithm & Serialization

1 User Input

User
(Human)

"Find the latest API changes, summarize them, and update my notes."

Function

Receive the user request and prepare it for processing.

Input

Natural language prompt (from user).

Output

Raw prompt text.

2 Tokenize / Encode

Tokenizer Agent
(Tokenizer Service)

Function

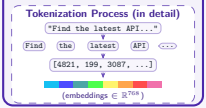
Tokenize prompt into IDs and compute dense embeddings.

Input

Raw prompt text.

Output

Token IDs + embedding vectors.



3 Intent Classifier

Intent Classifier Agent
(LLM / Classifier)

Function

Analyze contextual embeddings to classify user intent, extract domain entities (target API, destination notes), resolve dependencies, and evaluate confidence thresholds for task routing.

Input

Token embeddings + prompt context.

Output

Intent label(s) with confidence (research: 0.98, summarize: 0.95, write: 0.92) + entity slots.

4 Semantic Decomposer

Decomposer Agent
(Planning Agent)

Function

Break the classified intent into modular sub-tasks, establish execution ordering, and detect dependencies.

Input

Intent labels + context from Step 3.

Output

JSON-RPC task packet:

```
{"tasks": ["research",
"extract", "summarize",
"write"],
"approval_gate": true
}
```

5 Task DAG Output

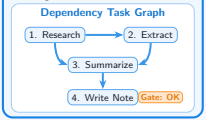
Tool Executor Agent
(Tooling Agent)

Function

Construct, validate, and execute the Directed Acyclic Graph (DAG) for dependency-ordered tool execution.

Input

Serialized task packet + tool bindings.



Tokenization Process (Step 2 – Detailed)

1. Raw Prompt

Input

"Find the latest API changes, summarize them, and update my notes."

(string)

2. Tokenization

Function

Split text into tokens (using BPE / WordPiece).

Output

`["Find", "the", "latest", "API", "changes", ...]`

(tokens)

3. Convert to IDs

Function

Map tokens to unique integers from vocabulary.

Output

`[4821, 199, 3087, 421, ...]`

(token IDs)

4. Create Embeddings

Function

Convert IDs to vector representations (e.g., 768-d).

Output

`[0.12, 0.45, -0.03, ...]`

(embedding vectors)

5. Pass to Intent Classifier

Function

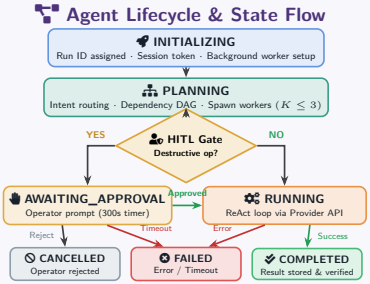
Embeddings + context → intent classification.

Output

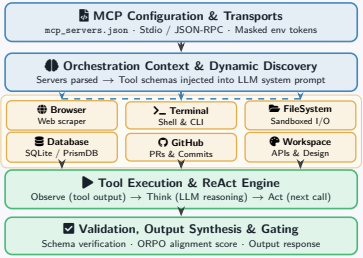
Intent label(s) + entities ("research", "summarize", ...).

07. Agent Lifecycle, MCP Integration & Runtime Flow in PrismSpace

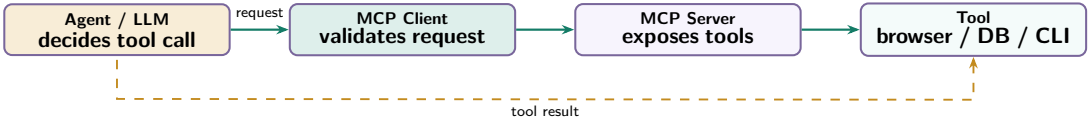
Agent Lifecycle & State Flow



MCP Tool Integration & Runtime Flow



How Agents Communicate with MCP




```
-> request:  {"jsonrpc":"2.0","id":7,"method":"request","params":{"name":"search","arguments":{"q":"MCP"}}}
<- response:  {"jsonrpc":"2.0","id":7,"result":{"content":[{"type":"text","text":"3 matches"}]}}
```

11. How the System Works: From Request to Result

Stage 1: Ingestion & Features

- **Capture request:** Parse user input (text/voice/API).
- **Clean & normalize:** Remove noise, handle missing data with NLP preprocessing.
- **Extract features:** Tokenize into semantic embeddings using transformer models.

Stage 2: Intelligence & DAG

- **Classify intent:** ML classifier identifies task type from 12 categories (99.6% accuracy).
- **Choose agents & models:** Route sub-tasks to specialized LLM agents.
- **Topological sorting:** Order DAG tasks by dependencies; execute only after all prerequisites finish.

Stage 3: Execution & Storage

- **Execute:** Run up to 3 parallel tasks using Model Context Protocol (MCP) tools.
- **Track progress:** Real-time telemetry, node state transitions, and execution logs.
- **Verify & store:** ORPO quality check, persist results to PrismDB.

Building & Executing the Task Graph $\mathcal{G}(V, E)$

1. User Goal Prompt
"Analyze news+CSV, email summary"

→

2. Split into Vertices V
Modular async tasks A, B, C, D

→

3. Connect Edges E
Map output return → input args

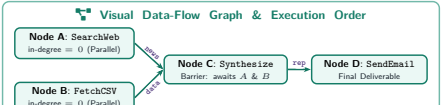
→

4. Topological Sort
Pick in-degree $= 0$; unlock dependents

Computer Science Explanation (By Concrete Example):

- **Vertices (V) = Sub-Task Functions (Async Call Units):**
  Node A: `news = await SearchWeb("Tesla news")` *[Researcher]*
  Node B: `data = await FetchCSV("metrics.csv")` *[Data Agent]*
  Node C: `rep = await Synthesize(news, data)` *[Writer Agent]*
  Node D: `await SendEmail(to="lead", body=rep)` *[Email Tool]*
- **Edges (E) = Data Pipes & Synchronization Barriers:**
  Edge (A, C) pipes return value `news`; edge (B, C) pipes `data`.
  Barrier: task C blocks until both promises A & B resolve.
- **Connection Basis:** C links to A & B because its inputs require their outputs; A and B are independent.
- **Topological Sorting (Kahn's method):** repeatedly select nodes with in-degree $= 0$, remove outgoing edges, and unlock dependents.
  This gives a valid dependency order and exposes independent work for parallel execution.
- **Execution:** A and B run in parallel; after both finish, C unlocks, then D runs.

Visual Data-Flow Graph & Execution Order



Parallel Thread 1: Worker 1 executes Node A → returns `news`
Parallel Thread 2: Worker 2 executes Node B → returns `data`
Join Barrier: Node C consumes `news` + `data` → outputs `rep`

Serial Finish: Node D consumes `rep` → sends email

Why Topological Sorting is Chosen for DAG Task Scheduling

1. Task Dependencies

- **Prerequisites first:** A task starts after the tasks it needs are finished.
- **Independent work:** Unrelated tasks can run in any order.

2. Finds Waiting Loops

- **Finds circular waiting:** Detects loops such as A waits for B, while B waits for A.
- **Stops safely:** Reports the problem instead of waiting forever.

3. Runs Work in Parallel

- **Ready tasks:** Tasks with no unfinished prerequisites can start together.
- **Safe hand-off:** The next task starts only when all required inputs arrive.

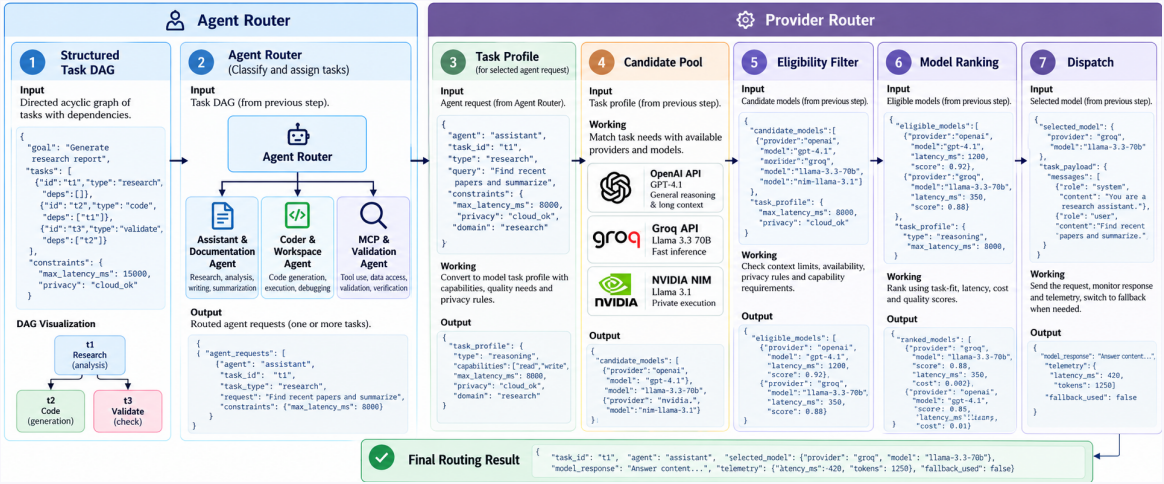
Simple Comparison: Topological Sort vs. Common Sorting

Method	Works With	How It Orders	Speed	Finds Loops	Parallel Work	Useful for Task Workflows
Topological Sort	Task dependency graph	**Prerequisites first**	$\mathcal{O}(V + E)$	Yes (In-degree)	Yes (Fan-out)	**Best choice:** Follows task dependencies and runs independent tasks together.
Merge Sort	List of values	**Smallest value first**	$\mathcal{O}(n \log n)$	No (N/A)	No (Serial 1D)	**Not suitable:** Sorts values, not task prerequisites.
Quick Sort	List of values	**Around a pivot value**	$\mathcal{O}(n \log n)$ avg	No (N/A)	No (Serial 1D)	**Not suitable:** Does not know which task must come first.
Heap Sort	Priority list	**Highest priority first**	$\mathcal{O}(n \log n)$	No (N/A)	No (Serial 1D)	**Not suitable:** Priority alone cannot enforce several prerequisites.
Bubble / Insertion	List of values	**Neighbor comparisons**	$\mathcal{O}(n^2)$	No (N/A)	No (Strict Serial)	**Not suitable:** Pairwise swaps cannot represent task dependencies.

Why This Matters for PrismSpace: Regular sorting puts numbers or words in order. PrismSpace must order tasks based on their prerequisites. **Topological sorting** follows those prerequisites, detects circular dependencies, and lets independent tasks run together.

Agent & Provider Router working

Routing goal: Convert a task profile into one suitable model request, then return the model response.

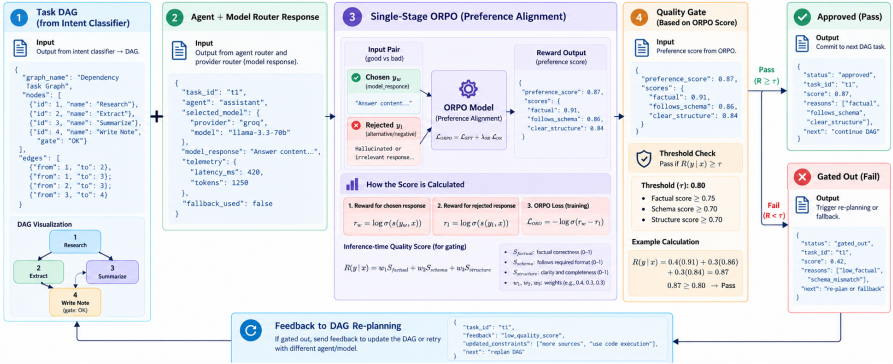


How ORPO Works: Preference Alignment and Quality Gating

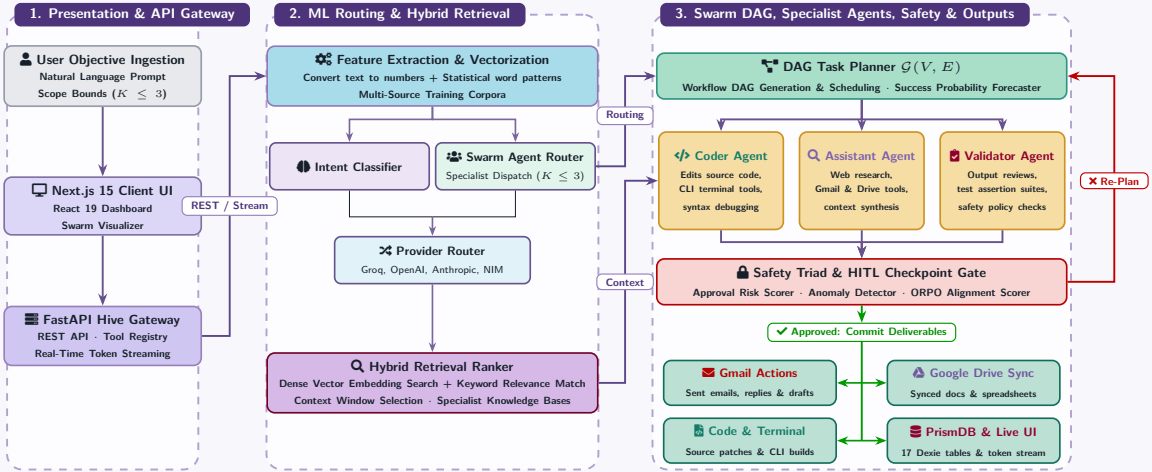
What is ORPO & Why is it Used Here as a Reward System?

- **What is ORPO? Odds Ratio Preference Optimization (ORPO)** is an integrated **single-stage reward and alignment algorithm**. Traditional RLHF (PPO) requires training a separate reward model and keeping a frozen reference model in GPU memory (heavy, slow, and unstable).
- **Why in PrismSpace?** In multi-agent swarms, LLMs can hallucinate invalid tools or malformed code. ORPO acts as an **internal reward filter**: it mathematically rewards compliant trajectories (y_w) and penalizes bad responses (y_l) during fine-tuning, eliminating the need for an external reward model.

End-to-End Pipeline: From Preference Pairs to Runtime Quality Gating



12. PrismSpace System Architecture & Multi-Model Swarm Workflow (Figure 1.2)



13. Flowchart: Multi-Agent Execution Flow (Figure 1.3)

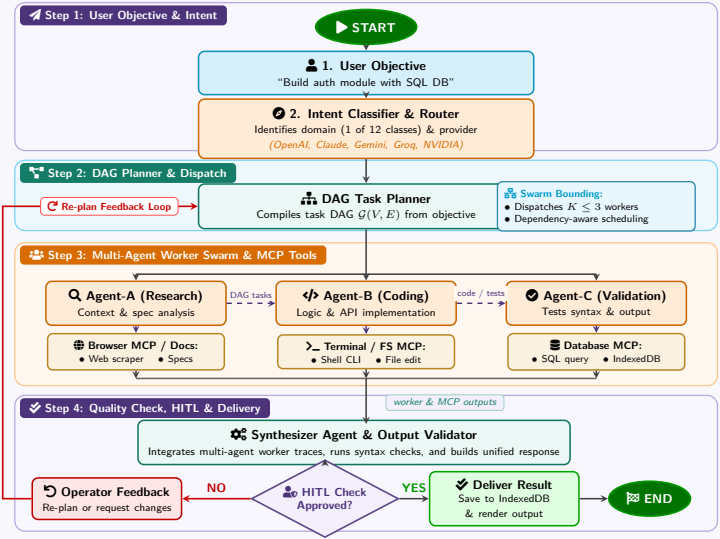
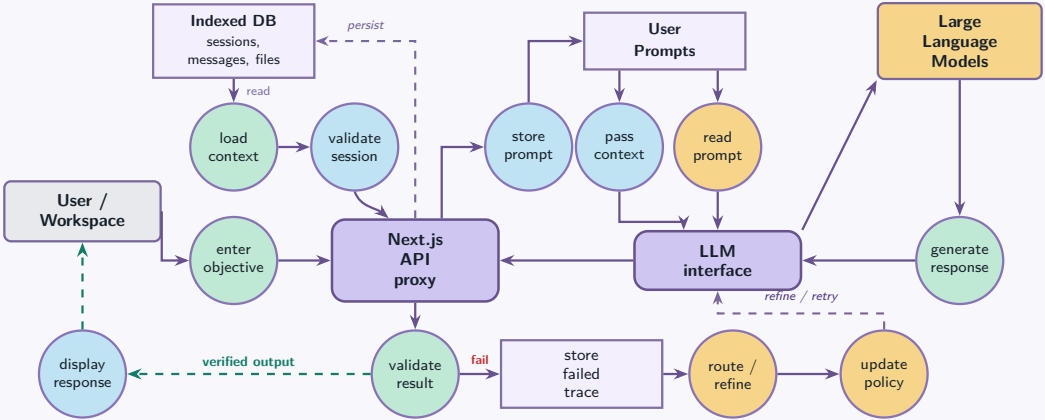


Figure 1.3: Simplified Multi-Agent Communication Flow in PrismSpace

07. PrismSpace Data Flow Diagram



Database Design

PrismDB schema, relationships, and entity–relationship structure

14. Database Schema

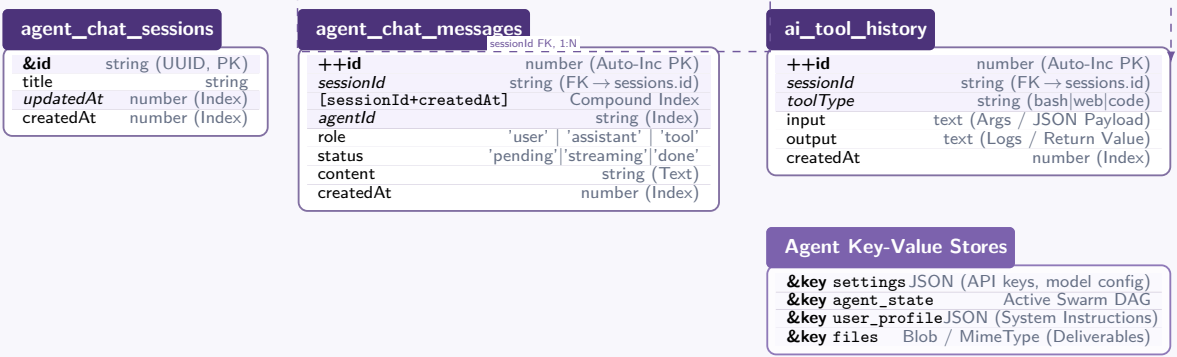
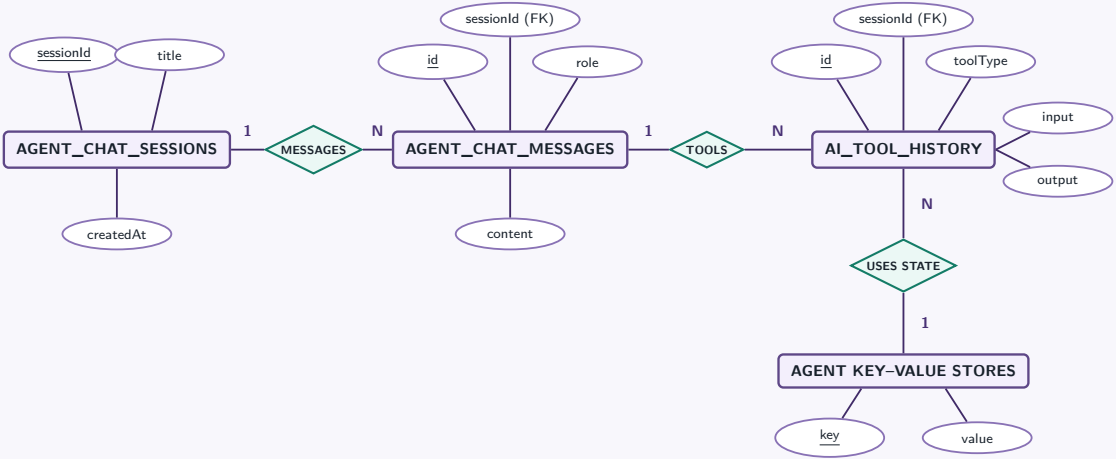


Figure 1.4: PrismDB - AI-Agent Browser IndexedDB Schema (Dexie.js v3) | **PK**: Primary Key **FK**: Foreign Key **Index**: B-tree index
`sessions.id` (Shared FK) → `messages.sessionId` & `tool_history.sessionId` | `useLiveQuery()` Reactive UI | ACID Dexie.js

15. Database ER Diagram



Project Requirements

Hardware and software needed to build and run PrismSpace

16. Hardware Requirements

- **CPU:** Multi-core processor for the Next.js client, FastAPI service, and local development tools.
- **GPU:** NVIDIA RTX 3050 or above for embedding generation, vector indexing, and model-training workloads.
- **Memory:** Minimum 16 GB RAM for running the frontend, backend, databases, and supporting services concurrently.
- **Storage:** SSD with sufficient capacity for the codebase, PrismDB data, logs, model checkpoints, and FAISS indexes.
- **Network:** Reliable internet connection for provider APIs, MCP integrations, and model downloads.
- **Security:** Secure environment-variable or secret-manager support for provider and tool credentials.

17. Software Requirements

- **Next.js 14 / React / TypeScript:** browser-based workspace for objectives, chat history, agent status, and approvals.
- **FastAPI / Python:** asynchronous backend framework for agent orchestration, REST endpoints, and runtime events.
- **Dexie.js / IndexedDB:** local ACID storage for sessions, messages, files, settings, and tool history.
- **Joblib / scikit-learn:** Python libraries for data preparation, utility functions, serialization, and general machine-learning support.
- **MCP JSON-RPC:** MCP (Model Context Protocol) uses JSON-RPC (JavaScript Object Notation–Remote Procedure Call) messages to standardize tool names, inputs, outputs, and errors for browser, terminal, filesystem, GitHub, database, and workspace tools.
- **Provider SDKs/APIs: NVIDIA NIM** provides optimized model-serving endpoints; the **Groq SDK** supports low-latency inference; and the **Anthropic Claude SDK/API**, OpenAI-compatible APIs, and similar provider clients connect the orchestrator to different language models.

Project Implementation

Frontend, backend, and training components

18. Frontend Implementation

- **Next.js 14:** React-based framework for building the browser application and managing frontend pages.
- **React:** component library for creating reusable user-interface elements.
- **TypeScript:** typed JavaScript used for safer and more maintainable frontend development.
- **Tailwind CSS:** utility-first styling framework for responsive layouts and consistent visual design.
- **Dexie.js, IndexedDB, and dexie-react-hooks:** browser storage and reactive data-access libraries for local application data.
- **Lucide React:** icon library used for interface symbols and controls.
- **Framer Motion and GSAP (GreenSock Animation Platform):** frontend animation libraries for smooth interface transitions.

19. Backend Implementation

- **FastAPI:** Python framework for building asynchronous backend services and application endpoints.
- **Python:** primary backend language for API services, integrations, data handling, and runtime coordination.
- **Uvicorn:** lightweight **ASGI (Asynchronous Server Gateway Interface)** server used to run FastAPI. ASGI connects asynchronous Python applications to web servers and supports non-blocking concurrent requests.
- **REST (Representational State Transfer):** API style for communicating with the frontend through structured HTTP requests and responses.
- **Pydantic:** schema validation and typed data models for request, response, configuration, and tool payloads.
- **MCP (Model Context Protocol) and JSON-RPC (JavaScript Object Notation–Remote Procedure Call):** standards for structured communication between the backend and external tools.
- **Provider SDKs/APIs:** integration layer for connecting the backend to external language-model and model-serving services.

20. Model Training Modules

- **Dataset preparation:** collects, cleans, and combines task traces and preference data using **Pandas**, **NumPy**, and **PyArrow**.
- **Intent classification:** categorizes user objectives using **scikit-learn** text features, classifiers, and **XGBoost**.
- **Agent routing:** selects specialist agents using shared **scikit-learn** pipelines and **XGBoost** classification support.
- **Model and provider routing:** compares task features and provider signals using **Pandas**, **NumPy**, **scikit-learn**, and **XGBoost**.
- **Performance prediction:** estimates latency and cost with **XGBoost** regression, supported by **scikit-learn** preprocessing.
- **Safety prediction:** estimates approval needs and detects unusual patterns with **scikit-learn**, **XGBoost**, and **Isolation Forest** utilities.
- **Retrieval:** uses **FAISS (Facebook AI Similarity Search)**, **NumPy**, and optional **sentence-transformers** embeddings for context search.
- **Preference alignment:** uses **PyTorch**, **Hugging Face Transformers**, **TRL (Transformer Reinforcement Learning)**, **PEFT**, and **Datasets** for ORPO training.
- **Runtime artifacts:** uses **Joblib**, **Safetensors**, and **Hugging Face Accelerate** to save, validate, and load trained components.

Artifact: a saved output from training, such as a trained model, vector index, or configuration file, that can be reused later. **FAISS (Facebook AI Similarity Search):** a library for quickly finding similar vectors. **PEFT (Parameter-Efficient Fine-Tuning):** adapts a model by training only a small set of additional parameters.

System Design

Modules, responsibilities, and system-level organization

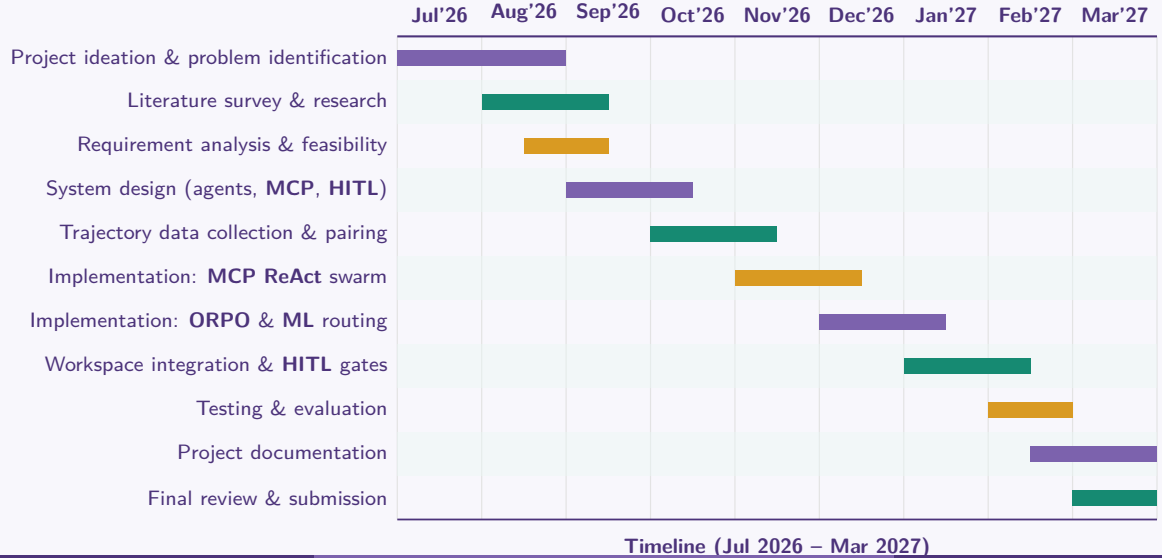
21. System Modules and Responsibilities

- **User and Session Module:** captures objectives, settings, conversation history, uploaded files, and session state.
- **Context and Routing Module:** classifies intent and selects the most suitable provider, model, and specialist agents.
- **DAG (Directed Acyclic Graph) Planning Module:** decomposes objectives into dependency-aware subtasks and schedules parallel work.
- **Agent Execution Module:** runs bounded **ReAct (Reasoning and Acting)** loops and invokes approved tools.
- **MCP (Model Context Protocol) Integration Module:** exposes filesystem, terminal, database, memory, and external-service tools through **JSON-RPC (JavaScript Object Notation–Remote Procedure Call)**.
- **Verification and Safety Module:** applies schema checks, **ORPO (Odds Ratio Preference Optimization)** preference gating, anomaly detection, and **HITL (Human-in-the-Loop)** approvals.
- **Persistence Module:** stores sessions, messages, artifacts, and errors in the local **PrismDB (browser IndexedDB database)**.

22. Conclusion

- **PrismSpace** converts a user's natural-language goal into smaller, organized tasks.
- Independent tasks can be executed by multiple agents in parallel.
- Standard tools are connected through the **Model Context Protocol (MCP)**.
- Human approval and local browser storage improve safety, privacy, and control.
- The platform provides a flexible foundation for future agent-based applications.

23. Gantt Chart



References

[1] **Cornalba F, Disselkamp C, Scassola D, Helf C (2024)** Multi-objective reward generalization: improving performance of Deep Reinforcement Learning for applications in single-asset trading. *Neural Computing and Applications*. DOI: 10.1007/s00521-023-09033-7

[2] **Schneckenreither M, Moser G (2025)** Average reward adjusted discounted reinforcement learning. *Neural Computing and Applications*. DOI: 10.1007/s00521-024-10620-5

[3] **Aina K, Ha S (2026)** Deep reinforcement learning for multi-agent coordination. *Artificial Life and Robotics*. DOI: 10.1007/s10015-025-01089-z

[4] **Rother D, Pajarinen J, Peters J, Weisswange TH (2025)** Open-ended coordination for multi-agent systems using modular open policies. *Autonomous Agents and Multi-Agent Systems*. DOI: 10.1007/s10458-025-09723-7

[5] **Liu D, Ren F, Yan J, Su G, Gu W, Kato S (2026)** Improving scalability of multi-agent deep reinforcement learning with suboptimal human knowledge. *Autonomous Agents and Multi-Agent Systems*. DOI: 10.1007/s10458-025-09729-1

[6] **Marzari L, Liu C, Donti PL, Marchesini E (2026)** ε-retraining reinforcement learning algorithms. *Autonomous Agents and Multi-Agent Systems*. DOI: 10.1007/s10458-026-09748-6

[7] **Wong A, Bäck T, Kononova AV, Plaat A (2023)** Deep multiagent reinforcement learning: challenges and directions. *Artificial Intelligence Review*. DOI: 10.1007/s10462-022-10299-x

[8] **Niu B, Luo B, Cui Y, Xu X, Zhao Y, Feng Y (2025)** HIAT: human-in-the-loop reinforcement learning with auxiliary task. *Artificial Intelligence Review*. DOI: 10.1007/s10462-025-11260-4

References

[9] **Lee J (2025)** InstructPatentGPT: training patent language models to follow instructions with human feedback. *Artificial Intelligence and Law*. DOI: 10.1007/s10506-024-09401-1

[10] **Zhuang S, Qiao Z, Zuccon G (2023)** Reinforcement online learning to rank with unbiased reward shaping. *Information Retrieval Journal*. DOI: 10.1007/s10791-022-09413-y

[11] **Ghazal TM, Awouda AE, Hasan MK, Rahman AHA, Islam S, Saeed RA, Elshafie H, Iqbal A, Hussein AH (2025)** Knowledge Learning for Securing Workflow Scheduling Algorithm in Mobile Edge Computing. *Mobile Networks and Applications*. DOI: 10.1007/s11036-025-02465-6

[12] **Barredo P, Puente J (2025)** Energy-aware cooperative multi-fitness evolutionary algorithm for workflow scheduling in cloud computing. *Natural Computing*. DOI: 10.1007/s11047-025-10023-y

[13] **Heirati A, Fartash M, Khalily-Dermany M (2025)** Optimized Task Scheduling in Fog-Cloud Computing Using Hybrid Deep Learning and Metaheuristic Algorithms. *Neural Processing Letters*. DOI: 10.1007/s11063-025-11819-w

[14] **Abdi S, Ashjaei M, Mubeen S (2024)** Cost-aware workflow offloading in edge-cloud computing using a genetic algorithm. *The Journal of Supercomputing*. DOI: 10.1007/s11227-024-06341-0

[15] **Xia Y, Zhu J, Zhu L (2023)** Dynamic role discovery and assignment in multi-agent task decomposition. *Complex & Intelligent Systems*. DOI: 10.1007/s40747-023-01071-x

[16] **Li J, Xing L, Zhong W, Cai Z, Hou F (2023)** Decision variable contribution based adaptive mechanism for evolutionary multi-objective cloud workflow scheduling. *Complex & Intelligent Systems*. DOI: 10.1007/s40747-023-01137-w

References

[17] **Yu X, Lin Y, Wang X, Han S, Lv K (2024)** GHQ: grouped hybrid Q-learning for cooperative heterogeneous multi-agent reinforcement learning. *Complex & Intelligent Systems*. DOI: 10.1007/s40747-024-01415-1

[18] **Xu C, Wang J, Zhu X, Yue Y, Zhou W, Liang Z, Wojtczak D (2024)** Decentralized multi-agent cooperation via adaptive partner modeling. *Complex & Intelligent Systems*. DOI: 10.1007/s40747-024-01421-3

[19] **Vyhmeister E, Castañé GG, Östberg PO, Thevenin S (2023)** A responsible AI framework: pipeline contextualisation. *AI and Ethics*. DOI: 10.1007/s43681-022-00154-8

[20] **Cavalcante Siebert L, Lupetti ML, Aizenberg E, Beckers N, Zgonnikov A, Veluwenkamp H, Abbink D, Giaccardi E, Houben GJ, Jonker CM, van den Hoven J, Forster D, Lagendijk RL (2023)** Meaningful human control: actionable properties for AI system development. *AI and Ethics*. DOI: 10.1007/s43681-022-00167-3

[21] **Yue Q, Tian L, Feng X, Yuan Z, Wei S, Xia J, Chen B, Song X, Yao Y (2026)** CPN-HRL: a hierarchical deep reinforcement learning approach for priority-aware task scheduling in CPN enabled by cloud-edge-end environments. *Journal of King Saud University Computer and Information Sciences*. DOI: 10.1007/s44443-026-00690-x

[22] **Skirzyński J, Jain Y, Lieder F (2023)** Automatic discovery and description of human planning strategies. *Behavior Research Methods*. DOI: 10.3758/s13428-023-02062-z

[23] **Zhang S, Zhao Z, Liu C, Qin S (2023)** Data-intensive workflow scheduling strategy based on deep reinforcement learning in multi-clouds. *Journal of Cloud Computing*. DOI: 10.1186/s13677-023-00504-9

[24] **Wang Z, Chen S, Bai L, Gao J, Tao J, Bond RR, Mulvenna MD (2023)** Reinforcement learning based task scheduling for environmentally sustainable federated cloud computing. *Journal of Cloud Computing*. DOI: 10.1186/s13677-023-00553-0

[25] **Subashree S, Rajakumaran M, Pushpa G, Senthilkumar S (2025)** Hybrid dolphin swarm sparrow search optimization based multi-objective cloud workflow scheduling. *Journal of Cloud Computing*. DOI: 10.1186/s13677-025-00828-8

Thank You!

Nobin Sijo | Pranav P | Prasanth P | Sankar Narayan S

Department of Computer Science & Engineering
College of Engineering, Karunagappally